CASO PRÁCTICO 1

Se pide
Partiendo de los siguientes diccionarios de inglés a otros idiomas (español, italiano, francés y alemán), crear un nuevo fichero donde cada palabra en inglés (clave) esté asociada a cada una de las traducciones con un separador.

Ejemplo:
1 - Descargar el fichero.
Lo primero que debe hacerse es descargar los diferentes diccionarios a una nueva carpeta (emplear, por ejemplo, comando ***wget***).
`wget` http://www.ilovelanguages.com/IDP/files/Spanish.txt
`wget` http://www.ilovelanguages.com/IDP/files/Italian.txt
`wget` http://www.ilovelanguages.com/IDP/files/French.txt
`wget` http://www.ilovelanguages.com/IDP/files/German.txt

Si la descarga de los archivos no funciona correctamente o el enlace está roto, descargar:
Diccionarios.zip

2 - *Merge* de ficheros.
Como Hadoop trabaja mejor con ficheros grandes, se hará un *merge* de los cuatro ficheros para obtener un único fichero diccionario. Por ejemplo, se puede emplear el comando ***CAT*** redirigiendo la salida a un nuevo fichero llamado “dictionary.txt”.

3 - Subir el fichero a HDFS.
Empleando alguno de los comandos de subida a HDFS, mover el fichero a una nueva carpeta en la ruta “/user/bigdata/mapreduce”, nombrándolo como “diccionario.txt”.

4 - Crear un fichero “Dictionary.java” propio.
Partiendo de los siguientes pantallazos, crear un fichero llamado “Dictionary.java”.
Importante: en el siguiente **enlace** se facilita el código que aparece en las capturas de pantalla en un archivo editable.
A_Dictionary_java.docx

```
bigdata@bigdata: ~/mapreduce
GNU nano 2.2.6                                  Archivo: Dictionary.java

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.KeyValueTextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class Dictionary {
```

MAPEO

```
public static class WordMapper extends Mapper<Text, Text, Text, Text>    {
    private Text word = new Text();
    public void map(Text key, Text value, Context context) throws IOException, InterruptedException    {
        StringTokenizer itr = new StringTokenizer(value.toString(),",");
        while (itr.hasMoreTokens())  {
            word.set(itr.nextToken());
            context.write(key, word);
        }
    }
}
```

REDUCCIÓN

```
public static class AllTranslationsReducer  extends Reducer<Text,Text,Text,Text>  {
    private Text result = new Text();
    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
        String translations = "";
        for (Text val : values) {
            translations += "|"+val.toString();
        }
        result.set(translations);
        context.write(key, result);
    }
}
```

MAIN

```
    public static void main(String[] args) throws Exception
    {
        Configuration conf = new Configuration();
        Job job = new Job(conf, "dictionary");
        job.setJarByClass(Dictionary.class);
        job.setMapperClass(WordMapper.class);
        job.setReducerClass(AllTranslationsReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setInputFormatClass(KeyValueTextInputFormat.class);
        FileInputFormat.addInputPath(job, new Path("/user/bigdata/mapreduce/diccionario.txt"));
        FileOutputFormat.setOutputPath(job, new Path("output"));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

5 - Compilar el fichero.

6 - Crear el “.jar".

7 - Ejecutar el proceso *MapReduce*.

8 - En este ejemplo, como se ha visto, no hay parámetros de entrada, ya que vienen indicados en el código.

9 - Devolver traducción de “pig”.

Solución:

Antes de nada, lo primero que hacemos es comprobar las versiones de **Java** y **Hadoop**.

```
bigdata@bigdata-IMF:~$ java -version
openjdk version "1.8.0_312"
OpenJDK Runtime Environment (build 1.8.0_312-8u312-b07-0ubuntu1~20.04-b07)
OpenJDK 64-Bit Server VM (build 25.312-b07, mixed mode)
bigdata@bigdata-IMF:~$ hadoop version
Hadoop 3.3.1
Source code repository https://github.com/apache/hadoop.git -r a3b9c37a397ad4188041dd80621bdeefc46885f2
Compiled by ubuntu on 2021-06-15T05:13Z
Compiled with protoc 3.7.1
From source with checksum 88a4ddb2299aca054416d6b7f81ca55
This command was run using /home/bigdata/hadoop/share/hadoop/common/hadoop-common-3.3.1.jar
bigdata@bigdata-IMF:~$
```

Editamos los archivos ***mapred-site.xml*** y ***yarn-site.xml*** para indicarle a **Hadoop** dónde están sus propios jars y cómo debe ejecutar **MapReduce** sobre **YARN**. Esto lo hacemos a través del terminal utilizando el editor **nano**, o bien editando el archivo directamente.

En ***mapred-site.xml*** añadimos una nueva propiedad que indica dónde encontrar las clases de **MapReduce**.

```
  GNU nano 4.8                                    /home/bigdata/hadoop/etc/hadoop/mapred-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
  <name>mapreduce.framework.name</name>
  <value>yarn</value>
</property>
<property>
  <name>mapreduce.application.classpath</name>
  <value>
    /home/bigdata/hadoop/share/hadoop/mapreduce/*,
    /home/bigdata/hadoop/share/hadoop/mapreduce/lib/*
  </value>
</property>
</configuration>
```

Y en ***yarn-site.xml*** introducimos en la propiedad ***yarn.application.classpath*** las rutas que usará cualquier job para encontrar las clases necesarias.

```
  GNU nano 4.8                                                    /home/bigdata/hadoop/etc/hadoop/yarn-site.xml
<?xml version="1.0"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<configuration>
 <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
 </property>
<property>
        <name>yarn.application.classpath</name>
        <value>
          /home/bigdata/hadoop/etc/hadoop,
          /home/bigdata/hadoop/share/hadoop/common/*,
          /home/bigdata/hadoop/share/hadoop/common/lib/*,
          /home/bigdata/hadoop/share/hadoop/hdfs/*,
          /home/bigdata/hadoop/share/hadoop/hdfs/lib/*,
          /home/bigdata/hadoop/share/hadoop/mapreduce/*,
          /home/bigdata/hadoop/share/hadoop/mapreduce/lib/*,
          /home/bigdata/hadoop/share/hadoop/yarn/*,
          /home/bigdata/hadoop/share/hadoop/yarn/lib/*
        </value>
</property>
</configuration>
```

Una vez configurado todo, iniciamos **HDFS** y **YARN**, y verificamos que los procesos necesarios se están ejecutando.

```
bigdata@bigdata-IMF:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-IMF]
bigdata@bigdata-IMF:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
bigdata@bigdata-IMF:~$ jps
47298 DataNode
49059 Jps
47525 SecondaryNameNode
48778 NodeManager
47146 NameNode
48299 ResourceManager
bigdata@bigdata-IMF:~$
```

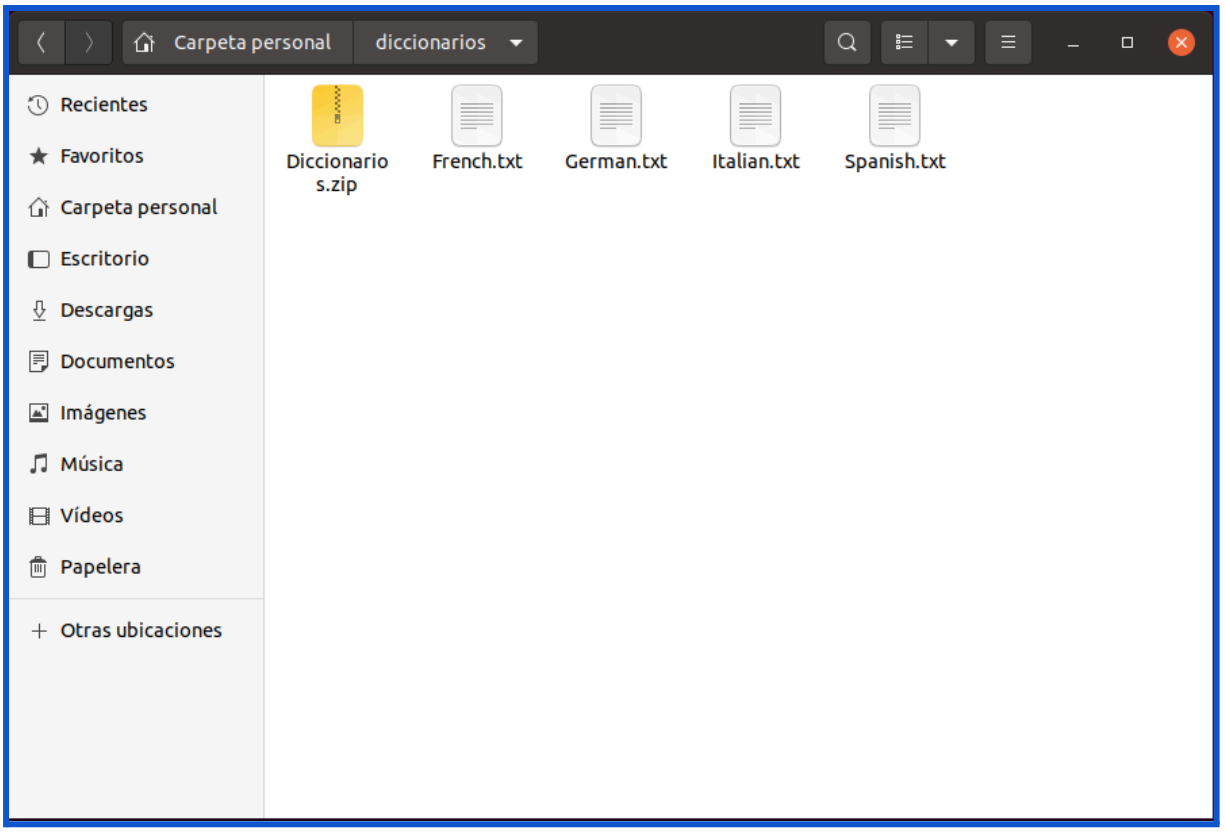
1. Descargar el fichero:

    a. Creamos la carpeta ***diccionarios*** y movemos el directorio a esta.

```
bigdata@bigdata-IMF:~$ mkdir -p ~/diccionarios && cd ~/diccionarios
bigdata@bigdata-IMF:~/diccionarios$
```

b. Copiamos en esa carpeta el archivo ***Diccionarios.zip*** y lo descomprimimos.

```
bigdata@bigdata-IMF:~/diccionarios$ unzip Diccionarios.zip
Archive:  Diccionarios.zip
  inflating: French.txt
  inflating: German.txt
  inflating: Italian.txt
  inflating: Spanish.txt
bigdata@bigdata-IMF:~/diccionarios$
```



c. Por último, normalizamos los finales de línea para evitar problemas en el merge de ficheros y en la lectura de **MapReduce**.

```
bigdata@bigdata-IMF:~/diccionarios$ for f in Spanish.txt Italian.txt French.txt German.txt; do
> sed -i 's/\r$//' "$f"
> done
bigdata@bigdata-IMF:~/diccionarios$
```

2. Merge de ficheros:

    a. Unimos los ficheros en uno y eliminamos las líneas que comiencen por #.

```
bigdata@bigdata-IMF:~/diccionarios$ cat Spanish.txt Italian.txt French.txt German.txt
| grep -v '^[[:space:]]*#' > dictionary.txt
bigdata@bigdata-IMF:~/diccionarios$
```

    b. Comprobamos el archivo e imprimimos las tres primeras líneas del mismo.

```
bigdata@bigdata-IMF:~/diccionarios$ ls -lh dictionary.txt
-rw-rw-r-- 1 bigdata bigdata 540K ago 26 20:39 dictionary.txt
bigdata@bigdata-IMF:~/diccionarios$ head -n 3 dictionary.txt
a        un, uno, una[Article]
aardvark         cerdo hormiguero
aardvark         oso hormiguero[Noun]
bigdata@bigdata-IMF:~/diccionarios$
```

3. Subir el fichero a **HDFS**:

    a. Creamos la carpeta dentro de **HDFS** y subimos el archivo diccionario.txt a la misma.

```
bigdata@bigdata-IMF:~/diccionarios$ hdfs dfs -mkdir -p /user/bigdata/mapreduce
bigdata@bigdata-IMF:~/diccionarios$ hdfs dfs -put -f dictionary.txt /user/bigdata/mapreduce/diccionario.txt
bigdata@bigdata-IMF:~/diccionarios$
```

    b. Por último, comprobamos que el archivo se encuentre correctamente en la carpeta.

```
bigdata@bigdata-IMF:~/diccionarios$ hdfs dfs -ls /user/bigdata/mapreduce
Found 1 items
-rw-r--r--   1 bigdata supergroup     552558 2025-08-27 23:11 /user/bigdata/mapreduce/diccionario.txt
bigdata@bigdata-IMF:~/diccionarios$
```

4. Crear un fichero ***Dictionary.java*** propio.

    a. Creamos la carpeta ***mapreduce*** y movemos el directorio a esta.

```
bigdata@bigdata-IMF:~/diccionarios$ mkdir -p ~/mapreduce && cd ~/mapreduce
bigdata@bigdata-IMF:~/mapreduce$
```

b. Creamos el fichero ***Dictionary.java*** usando **heredoc** y después comprobamos que se encuentre en la carpeta.

```
bigdata@bigdata-IMF:~/mapreduce$ cat > Dictionary.java <<'EOF'
> import java.io.IOException;
> import java.util.StringTokenizer;
>
> import org.apache.hadoop.conf.Configuration;
> import org.apache.hadoop.fs.Path;
> import org.apache.hadoop.io.Text;
> import org.apache.hadoop.mapreduce.Job;
> import org.apache.hadoop.mapreduce.Mapper;
> import org.apache.hadoop.mapreduce.Reducer;
> import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
> import org.apache.hadoop.mapreduce.lib.input.KeyValueTextInputFormat;
> import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
>
> public class Dictionary {
>
>   public static class WordMapper extends Mapper<Text, Text, Text, Text>{
>     private Text token = new Text();
>
>     public void map(Text key, Text value, Context context) throws IOException, InterruptedException {
>       StringTokenizer itr = new StringTokenizer(value.toString(), ",");
>       while (itr.hasMoreTokens()) {
>         token.set(itr.nextToken().trim());
>         if (token.getLength() > 0) {
>           context.write(key, token);
>         }
>       }
>     }
>   }
```

```
>
>   public static class AllTranslationsReducer extends Reducer<Text,Text,Text,Text> {
>     private Text result = new Text();
>
>     public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
>       StringBuilder sb = new StringBuilder();
>       for (Text v : values) {
>         sb.append("|").append(v.toString());
>       }
>       result.set(sb.toString());
>       context.write(key, result);
>     }
>   }
>
>   public static void main(String[] args) throws Exception {
>     Configuration conf = new Configuration();
>
>     Job job = Job.getInstance(conf, "dictionary");
>     job.setJarByClass(Dictionary.class);
>
>     job.setMapperClass(WordMapper.class);
>     job.setReducerClass(AllTranslationsReducer.class);
>
>     job.setOutputKeyClass(Text.class);
>     job.setOutputValueClass(Text.class);
>
>     job.setInputFormatClass(KeyValueTextInputFormat.class);
>
>     FileInputFormat.addInputPath(job, new Path("/user/bigdata/mapreduce/diccionario.txt"));
>     FileOutputFormat.setOutputPath(job, new Path("output"));
>
>     System.exit(job.waitForCompletion(true) ? 0 : 1);
>   }
> }
> EOF
bigdata@bigdata-IMF:~/mapreduce$
```

```
bigdata@bigdata-IMF:~/mapreduce$ ls -1
Dictionary.java
bigdata@bigdata-IMF:~/mapreduce$
```

c. Creamos el directorio ***classes*** y compilamos el código ***Dictionary.java***.

```
bigdata@bigdata-IMF:~/mapreduce$ mkdir -p classes
bigdata@bigdata-IMF:~/mapreduce$ javac -classpath "$(hadoop classpath)" -d classes Dictionary.java
bigdata@bigdata-IMF:~/mapreduce$
```

d. Empaquetamos el **JAR** resultante y comprobamos que exista:

```
bigdata@bigdata-IMF:~/mapreduce$ jar -cvf dictionary.jar -C classes/ .
manifiesto agregado
agregando: Dictionary$WordMapper.class(entrada = 1711) (salida = 745)(desinflado 56%)
agregando: Dictionary.class(entrada = 1590) (salida = 847)(desinflado 46%)
agregando: Dictionary$AllTranslationsReducer.class(entrada = 1852) (salida = 801)(desinflado 56%)
bigdata@bigdata-IMF:~/mapreduce$
```

```
bigdata@bigdata-IMF:~/mapreduce$ ls -lh dictionary.jar
-rw-rw-r-- 1 bigdata bigdata 3,1K ago 28 14:18 dictionary.jar
bigdata@bigdata-IMF:~/mapreduce$
```

5. Ejecutamos el **Job Map Reduce**:

a. Borramos el directorio de salida para evitar errores.

```
bigdata@bigdata-IMF:~/mapreduce$ hdfs dfs -rm -r -f output
bigdata@bigdata-IMF:~/mapreduce$
```

b. Ejecutamos el **Job MapReduce** desde el archivo **JAR**.

```
bigdata@bigdata-IMF:~/mapreduce$ hadoop jar dictionary.jar Dictionary
2025-08-28 14:27:04,993 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2025-08-28 14:27:05,464 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool in
terface and execute your application with ToolRunner to remedy this.
2025-08-28 14:27:05,483 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/bigdata/.st
aging/job_1756382857855_0001
2025-08-28 14:27:05,795 INFO input.FileInputFormat: Total input files to process : 1
2025-08-28 14:27:05,919 INFO mapreduce.JobSubmitter: number of splits:1
2025-08-28 14:27:06,157 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1756382857855_0001
2025-08-28 14:27:06,157 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-08-28 14:27:06,445 INFO conf.Configuration: resource-types.xml not found
2025-08-28 14:27:06,447 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2025-08-28 14:27:06,765 INFO impl.YarnClientImpl: Submitted application application_1756382857855_0001
2025-08-28 14:27:06,911 INFO mapreduce.Job: The url to track the job: http://bigdata-IMF:8088/proxy/application_1756382857855_0001/
2025-08-28 14:27:06,917 INFO mapreduce.Job: Running job: job_1756382857855_0001
2025-08-28 14:27:15,133 INFO mapreduce.Job: Job job_1756382857855_0001 running in uber mode : false
2025-08-28 14:27:15,136 INFO mapreduce.Job:  map 0% reduce 0%
2025-08-28 14:27:21,269 INFO mapreduce.Job:  map 100% reduce 0%
2025-08-28 14:27:26,332 INFO mapreduce.Job:  map 100% reduce 100%
2025-08-28 14:27:26,380 INFO mapreduce.Job: Job job_1756382857855_0001 completed successfully
2025-08-28 14:27:26,527 INFO mapreduce.Job: Counters: 54
```

```
        File System Counters
                FILE: Number of bytes read=611536
                FILE: Number of bytes written=1769043
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=552683
                HDFS: Number of bytes written=484821
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=3037
                Total time spent by all reduces in occupied slots (ms)=2946
                Total time spent by all map tasks (ms)=3037
                Total time spent by all reduce tasks (ms)=2946
                Total vcore-milliseconds taken by all map tasks=3037
                Total vcore-milliseconds taken by all reduce tasks=2946
                Total megabyte-milliseconds taken by all map tasks=3109888
                Total megabyte-milliseconds taken by all reduce tasks=3016704
```

```
        Map-Reduce Framework
                Map input records=23798
                Map output records=25078
                Map output bytes=561374
                Map output materialized bytes=611536
                Input split bytes=125
                Combine input records=0
                Combine output records=0
                Reduce input groups=13358
                Reduce shuffle bytes=611536
                Reduce input records=25078
                Reduce output records=13358
                Spilled Records=50156
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=101
                CPU time spent (ms)=1180
                Physical memory (bytes) snapshot=364990464
                Virtual memory (bytes) snapshot=5032001536
                Total committed heap usage (bytes)=230821888
                Peak Map Physical memory (bytes)=234708992
                Peak Map Virtual memory (bytes)=2512629760
                Peak Reduce Physical memory (bytes)=130281472
                Peak Reduce Virtual memory (bytes)=2519371776
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=552558
        File Output Format Counters
                Bytes Written=484821
bigdata@bigdata-IMF:~/mapreduce$
```

c. Comprobamos el directorio de salida:

```
bigdata@bigdata-IMF:~/mapreduce$ hdfs dfs -ls output
Found 2 items
-rw-r--r--   1 bigdata supergroup          0 2025-08-28 14:27 output/_SUCCESS
-rw-r--r--   1 bigdata supergroup     484821 2025-08-28 14:27 output/part-r-00000
bigdata@bigdata-IMF:~/mapreduce$
```

6. Observar resultados:

a. Observamos las primeras líneas del archivo de salida.

```
bigdata@bigdata-IMF:~/mapreduce$ hdfs dfs -head output/part-r-00000
AIDS    |AIDS
Africa  |Afrika
Alaska  |Alaska
American        |americano
American Dollar |amerikanische Dollar (m)
Ann     |Anna
Antarctica      |Antarktika
April   |April (m)|abril
Argentina       |Argentinien
Asia    |Asien (n)
August  |agosto|August (m)
Australia       |Australien
Australian      |Australier (m)
Bolivia |Bolivien
Brazil  |Brasilien
Brazilian       |brasilianisch|Brasilianer (m)
British |englisch|britisch
CD      |Kompakte Scheibe (f)
Cable News Network      |CNN
Canada  |Kanada
Canadian        |Kanadier (m)|kanadisch
Capricorn       |Steinbock (m)
Caribbean Sea   |Karibische See (f)
Carolyn |Carolyn
Chicken |Huhn (n)
Chile   |Chile
Chinese |chinesisch|Chinese (pl)
Chinese food    |chinesische Speise (f)
Christian       |christlich
Christine       |Christine
Christmas       |la Navidad|Weihnachten (n)|Christfest (n)
Christmas day   |Weihnachtstag (m)
Christmas eve   |Weihnachtsvorabend (m)
Christopher     |Kristophe
Costa Rica      |Kosta-Rika
DOS     |DOS
David   |David
Dear (...)      |geehrte (...)|liebe (...)
December        |diciembre|Dezember
Donald  |Donald
Donna   |Donna
Easter  |Ostern (n)
Ecuador |Equador
bigdata@bigdata-IMF:~/mapreduce$
```

b. Filtramos las líneas con una clave específica para encontrar la palabra ***pig***.

```
bigdata@bigdata-IMF:~/mapreduce$ hdfs dfs -cat output/part-r-00000 | grep -P '^pig\t'
pig     |el puerco|Schwein (n)|el chancho[Noun]|cochon[Noun]
bigdata@bigdata-IMF:~/mapreduce$
```